

Suivi du projet

Code

Architecture globale

Nous avons commencé le projet en partant sur une architecture MVC en trois couches : **Model**, **View** et **Controller**. Suite au cours théorique du 3 mars, nous avons opté pour ajouter des couches **Repository** et **Service** qui y ont été brièvement décrites.

Nous sommes encore flous sur la séparation des responsabilités de certaines couches, en particulier Model, Controller et Service.

Beaucoup de temps a été investi à faire du *refactoring* pendant cette première itération. Nous avons compris l'*extreme programming* comme nous disant de nous lancer tête baissée dans le code sans planification et avons codé comme ça. Mais des problèmes d'architecture et des désaccords entre les membres sur la façon de structurer le programme ont vite conduit à un code chaotique.

Repository

Le Repository est défini principalement par des **interfaces** (BugemonSpeciesRepository, ItemRepository, ...). Ces interfaces permettront à l'avenir de changer l'implémentation du stockage et du chargement de données, notamment en passant à une base de données, sans impacter le reste du code.

Une première ébauche de stockage de données avait été faite avant le cours du 3 mars avec des classes statiques (BugemonDatabase, AbilityDatabase, ...). Des problèmes sont vite survenus dans les tests unitaires avec cette approche, nous montrant l'importance de faire de l'**injection de dépendances**. Ceci était la motivation principale pour revoir l'architecture du programme et introduire la couche Repository.

A l'heure actuelle le Repository n'est pas encore bien intégré au reste du code. Il reste beaucoup d'artefacts d'anciennes architectures qui devront être enlevés petit à petit.

Service

La couche Service est encore très rudimentaire ; elle ne contient qu'un service Bugémon.

Comme l'architecture globale du projet est encore floue et constamment en mouvement, l'injection de dépendances n'est pas encore utilisée dans les trois couches MVC (on ne sait pas encore où instancier les services). Une classe statique ServiceLoader se charge de délivrer les services pour l'instant.

Avancée des histoires

Dès l'attribution des histoires par le client nous étions unanimes sur le fait que nous ne pourrions pas tout implémenter. Nous avons tout de suite décidé de ne pas faire l'histoire 9 : Exploration de la tour du NO. Malgré ça la tâche restait quand même trop lourde.

Surprenamment beaucoup de fonctionnalités ont quand même pu être implémentées. Mais plusieurs membres trouvent que l'architecture du code a été sacrifiée en faveur d'une implémentation rapide des fonctionnalités.

Nous avons fortement revu à la hausse nos estimations de temps pour l'itération 2.